FICHE · S10 4 MA BASE JSON

Semaine 10 · Exercice 4 — Ma base JSON

Module ICT 291 · Développer des interfaces utilisateurs (UI)

Durée : période 3 (étapes 1 à 5, 40 minutes), puis à la maison avant la semaine 11 (étapes 6 à 8, environ 40 minutes) · seul, avec votre brief · parcours essentiel et avancé

Où sont les fichiers : dans le pack, dossier `02-exercices/s10/4-ma-base-json/depart/`

Mandat

Votre persona attend une application qui affiche de vrais contenus, pas trois cartes de démonstration. Votre mission : écrire `data.json`, le fichier qui contiendra au moins 30 fiches pour votre projet, à partir de votre brief.

Pourquoi

La semaine prochaine, votre page lira ce fichier et affichera une carte par fiche. Une étiquette qui manque ici sera une information qui manque à l'écran. Une étiquette écrite de deux façons cassera votre filtre en semaine 12. Et la note 3 demande un fichier **personnel** d'au moins 30 fiches.

Ce que vous saurez faire

- Déduire de votre brief la liste des étiquettes, avec la méthode en 6 questions de l'exercice 3.
- Écrire 5 fiches à la main, sans faute.
- Faire compléter le fichier par une IA, puis contrôler et relire sa réponse.
- Enregistrer une version nommée de votre travail sur GitHub.

Les mots

- **Brief** : le document de la semaine 5 qui décrit votre application, ses écrans et ce que chaque écran affiche.
- **JSON** : une manière d'écrire des fiches dans un simple fichier texte, qui se termine par `.json`.
- **Fiche** : tout ce que l'on sait sur une chose (un lieu, un objet, un événement), entre accolades `{ }`.
- **Étiquette** : le nom d'une case de la fiche, entre guillemets doubles, par exemple `"nom"`. Une étiquette avec sa valeur s'appelle une *propriété*.
- **Valeur** : ce qui est rangé dans la case. Un texte entre guillemets doubles, un nombre sans guillemets, ou un vrai/faux écrit `true` ou `false`, sans guillemets.
- **Pile** : la liste de toutes les fiches, entre crochets `[ ]`, séparées par des virgules.
- **Valide** : le fichier respecte les règles d'écriture JSON. **Cohérent** : toutes les fiches portent les mêmes étiquettes, avec la même sorte de valeur.

- **Racine du dépôt** : le dossier principal de votre projet, celui qui contient `index.html`.
- **Commit** : une version nommée de votre projet, comme « Nommer la version actuelle » dans l'historique de Google Docs. On peut toujours y revenir.
- **Synchroniser** : envoyer vos commits sur GitHub, pour qu'ils soient sauvegardés en ligne.

Fichiers fournis

Dans le dossier `depart/` de cet exercice :

- `structure.exemple.json` : 2 fiches de Phare, l'application exemple du cours (les endroits calmes encore ouverts ce soir, près de chez soi). Il montre **la forme** du fichier, pas vos étiquettes.
- `etiquettes.md` : le tableau à remplir à l'étape 1.
- `controle-json.html` : le même outil de contrôle que dans les exercices 1 à 3.

> **NOTE**
> Données personnelles. — Ne collez jamais de données personnelles (noms, téléphones, adresses, notes) dans un site en ligne, même un validateur. Ce que vous collez part sur un serveur que vous ne connaissez pas. Pour votre projet, utilisez `controle-json.html` : il fonctionne sans Internet et rien ne sort de votre ordinateur.

Étapes

1. Du brief aux étiquettes

Dans le Finder ou l'Explorateur, copiez `etiquettes.md` dans le dossier de votre projet, à côté de `index.html`. **Dans VS Code**, ouvrez-le, puis relisez la section « Contenu de chaque écran » de votre brief. Appliquez la méthode en 6 questions de l'exercice 3 : ce qu'on affiche, ce qui sert à filtrer, ce qui sert à trier, la sorte de chaque valeur, ce qui est unique. Pour chaque étiquette, ajoutez une ligne au tableau : l'étiquette, l'écran qui l'affiche, à quoi elle sert, la sorte de valeur (texte, nombre, vrai/faux), si elle est unique, un exemple. Écrivez chaque étiquette en camelCase : minuscules, sans accent ni espace, et pour deux mots une majuscule au second, par exemple `prixMoyen`.

C'est réussi si : chaque ligne, sauf `id`, sert à afficher, filtrer ou trier, et aucun écran ne montre une information absente du tableau.

2. Créer data.json

Dans VS Code, dans la colonne de gauche, survolez le nom de votre projet, cliquez sur l'icône « Nouveau fichier », tapez `data.json`, puis Entrée.

C'est réussi si : `data.json` apparaît **au même niveau** que `index.html`, pas dans un sous-dossier.

3. Une première fiche

Dans `data.json`, écrivez `[`, puis votre première fiche, avec toutes les étiquettes de `etiquettes.md`, puis `]`. Prenez la forme de `structure.exemple.json`, mais vos étiquettes et votre contenu. Double-cliquez sur `controle-json.html`, collez tout le fichier, cliquez sur « Contrôler ».

C'est réussi si : l'outil affiche « Valide : 1 fiche ».

4. Cinq fiches à la main

Dans `data.json`, ajoutez les fiches 2 à 5, à la main. Pensez à la virgule après chaque `}`, sauf après la dernière. Variez les vrai/faux : au moins un `true` et un `false`. Contrôlez.

C'est réussi si : l'outil affiche « Valide : 5 fiches, toutes avec les mêmes étiquettes. »

5. Premier commit

Dans VS Code, cliquez sur l'icône Contrôle de code source dans la barre de gauche (trois ronds reliés). Dans la case « Message », écrivez `Ajoute data.json (5 fiches écrites à la main)`. Cliquez sur « Valider » (« Commit » si VS Code est en anglais), puis sur « Synchroniser les modifications ».

C'est réussi si : sur la page GitHub de votre dépôt, `data.json` apparaît avec ce message.

À la maison, avant la semaine 11

Les étapes 6 à 8 se font chez vous, avant le cours de la semaine 11 (environ 40 minutes). Au début de la semaine 11, l'enseignant contrôle en 5 minutes que votre `data.json` est vert avec au moins 30 fiches, et que GitHub montre les deux commits.

6. L'IA complète jusqu'à 30

Dans l'outil d'IA que vous utilisez pour le cours, collez le prompt ci-dessous, en remplaçant la ligne entre parenthèses par tout le contenu de votre `data.json` :

```
Je suis apprenti·e médiamaticien·ne. Voici mon fichier JSON, avec mes 5 premières fiches :

(collez ici tout votre data.json)

Recopiez mes 5 fiches sans les changer, puis ajoutez 25 nouvelles fiches (id de 6 à 30), dans
un seul tableau JSON.
Gardez exactement les mêmes étiquettes, dans le même ordre, sans en ajouter.
Contenu crédible pour la Suisse romande. Aucun nom de personne réelle.
Variez les valeurs vrai/faux : environ un tiers de false.
Guillemets droits ", pas de virgule après la dernière fiche, pas de commentaire.
Répondez d'abord par le tableau seul, de [ à ]. Puis, séparément, en 3 lignes : comment vous
avez varié les fiches, et les 2 fiches que je devrais vérifier.
```

Copiez **seulement** ce qui va du `[` au `]`. Dans `data.json`, sélectionnez tout et collez à la place. Contrôlez.

C'est réussi si : l'outil affiche « Valide : 30 fiches », et vos 5 fiches sont toujours les 5 premières, sans changement. S'il affiche une faute, corrigez-la comme dans l'exercice 1. Ne redemandez pas tout à l'IA.

7. Relire

Dans `data.json`, lisez 5 fiches au hasard, plus les 2 que l'IA vous a signalées. Pour chacune : le nom est-il crédible ? Le lieu existe-t-il ? Y a-t-il un doublon ? Un nom de camarade ? Dans l'outil, regardez la colonne « Valeurs différentes » : `true` et `false` doivent apparaître tous les deux.

C'est réussi si : vous avez corrigé ou supprimé au moins une fiche douteuse, et l'outil reste vert.

8. Deuxième commit

Dans VS Code, Contrôle de code source : message `Complète data.json à 30 fiches, contrôlé`, puis « Valider » et « Synchroniser les modifications ».

C'est réussi si : la page GitHub de votre dépôt montre deux commits pour `data.json`.

Réussi si

- `data.json` est à la racine du dépôt, à côté de `index.html`.
- L'outil affiche « Valide : N fiches, toutes avec les mêmes étiquettes. », avec N d'au moins 30.
- Chaque étiquette de `etiquettes.md`, sauf `id`, sert à afficher, filtrer ou trier sur un écran du brief.
- Deux commits sont visibles sur GitHub.

Si ça bloque

- **Je ne sais pas quelles étiquettes choisir.** Reprenez les 6 questions de l'exercice 3. Commencez par votre wireframe de l'écran Liste : chaque information visible sur une carte est une étiquette. Ajoutez `id`. Cela suffit pour commencer.
- **L'IA a ajouté une étiquette (`image`, `latitude`, `note`).** L'outil la signale. Supprimez-la de toutes les fiches, ou redemandez à l'IA « sans ajouter d'étiquette ». N'ajoutez pas une étiquette qui ne sert ni à afficher, ni à filtrer, ni à trier.
- **L'outil dit « Le texte commence par ``` ».** Vous avez copié les balises de code de l'IA. Supprimez la première et la dernière ligne.
- **L'outil dit « Du texte continue après la fin du fichier ».** Vous avez copié l'explication de l'IA. Supprimez tout ce qui suit le dernier `]`.
- **Votre quota d'IA est épuisé.** Changez d'outil, avec le même prompt. Personne ne paie un abonnement pour ce cours.
- **« Synchroniser » est grisé.** Le commit n'a pas été fait : vérifiez que vous avez cliqué sur « Valider » après avoir écrit le message.

Avec l’IA

L'IA écrit vite, mais elle ne connaît pas votre brief. Elle ajoute des étiquettes « au cas où », écrit parfois `"true"` entre guillemets, colle des balises ``` autour du tableau et invente des lieux. Ce n'est pas elle qui décide que le fichier est juste : c'est l'outil de contrôle pour l'écriture, et vous pour le contenu. Les 5 premières fiches sont écrites par vous, et l'IA ne fait que les prolonger. Ne lui donnez jamais de vraies données de personnes : votre prompt ne contient que vos 5 fiches, inventées ou publiques.

Pour les rapides

Parcours avancé : mêmes étapes, puis un ou plusieurs de ces points. Ils ne sont pas notés en plus.

- **Un horaire détaillé.** Si votre écran Fiche affiche des horaires, remplacez le texte `horaire` par une sous-fiche, dans **toutes** les fiches : `"horaire": { "ouverture": "07:00", "fermeture": "22:00" }`. L'outil affiche alors « fiche dans la fiche » pour cette étiquette.
- **Des mots-clés.** Seulement si votre brief prévoit un filtre par mot, ajoutez `"tags": ["calme", "wifi"]` : une liste de textes entre crochets.
- **Préparer la semaine 11.** Créez à côté de `data.json` deux fichiers : `vide.json`, qui contient seulement `[]`, et `casse.json`, une copie de `data.json` avec une virgule en trop. Ils serviront à tester ce que votre page affiche quand il n'y a rien, ou quand le fichier est cassé.
- **40 fiches**, si votre filtre doit montrer des résultats variés.